Gate 01 — Foundation

Status: Complete **Preconditions:** None **Unlocks:** Gate 02 (Data Layer)

Purpose

Establish the project skeleton so all subsequent gates have a working, containerized, CI-verified environment to build into. No domain logic in this gate — pure infrastructure.

Deliverables

- Django project initialized with split settings (`settings/base.py` , `settings/dev.py` , `settings/prod.py`)
- Docker Compose stack: `web` (Django + Gunicorn), `worker` (Celery, same image), `beat` (Celery Beat), `db` (PostgreSQL + PostGIS), `redis` , `minio` (dev only)
- GitHub Actions CI: lint (`ruff`), type check (`mypy --strict` on app code), unit tests (`pytest`) on every push; build check on PRs
- `Apache-2.0` license file committed
- `.env.example` committed; `.env` in `.gitignore`
- `GET /api/v1/health/` endpoint returning `{"status": "ok", "version": "..."}` — no auth required
- `pytest` configured to run against a real PostGIS-enabled test database (not mocked)
- `pre-commit` hooks: ruff, mypy

User Stories

BE-01-1: Health Check Endpoint

As a deployment operator, **I want** `GET /api/v1/health/` to return 200 with version info, **so that** load balancers and monitoring can verify the service is running.

DRF endpoint: `GET /api/v1/health/` **Auth required:** None **Response:**

```
{
  "status": "ok",
  "version": "0.1.0",
  "database": "connected",
  "cache": "connected"
}
```

Acceptance Criteria:

Given the Django application is running **When** `GET /api/v1/health/` is called with no authentication **Then** the response is HTTP 200 with `status: "ok"` and a `version` string

Given the database is unreachable **When** `GET /api/v1/health/` is called **Then** the response is HTTP 503 with `database: "error"` (does not crash)

Technical Tasks

- Initialize Django project with `django-admin startproject`
- Configure split settings with `DJANGO_SETTINGS_MODULE` environment variable
- Write `docker-compose.yml` with all six services; use named volumes for `db` and `redis` data
- Enable PostGIS extension in the `db` service via init SQL (`CREATE EXTENSION IF NOT EXISTS postgis;`)
- Configure `pytest.ini` / `pyproject.toml` with `pytest-django` , `DJANGO_SETTINGS_MODULE=settings.test` , `--reuse-db` for speed
- Configure `ruff` for Python 3.12+, line length 100, select E/F/W/I/N/UP
- Configure `mypy` with `django-stubs` and `djangorestframework-stubs`
- Set up GitHub Actions workflow file `.github/workflows/ci.yml`
- Commit `LICENSE` (Apache-2.0 text), `README.md` (one-paragraph project description), `.env.example`
- Install core dependencies: `django` , `djangorestframework` , `psycopg2-binary` , `django-environ` , `gunicorn` , `celery` , `redis` , `django-storages` , `boto3` , `Pillow`

Out of Scope

- Any data models (Gate 02)
- Auth configuration beyond Django's defaults (Gate 03)
- Frontend (Gates 05+)
- Any domain logic

Gate Exit Criteria

Before marking Gate 01 complete:

1. `docker-compose up` starts cleanly with no errors
2. `pytest` passes against a PostGIS-enabled database
3. GitHub Actions CI passes on a clean push
4. `GET /api/v1/health/` returns 200
5. Invoke **@code-quality-reviewer** on the project structure and CI configuration

---

Reconciliation

Implementation reconciled on 2026-04-16. All acceptance criteria met, all deliverables completed. See gate-01-reconciliation.md for full details including deviations, additions, and spec update recommendations.